JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class IsoChronology

java.lang.Object
    java.time.chrono.AbstractChronology
        java.time.chrono.IsoChronology

All Implemented Interfaces:

Serializable, Comparable<Chronology>, Chronology

```
public final class IsoChronology
extends AbstractChronology
implements Serializable
```

The ISO calendar system.

This chronology defines the rules of the ISO calendar system. This calendar system is based on the ISO-8601 standard, which is the *de facto* world calendar.

The fields are defined as follows:

- era - There are two eras, 'Current Era' (CE) and 'Before Current Era' (BCE).
- year-of-era - The year-of-era is the same as the proleptic-year for the current CE era. For the BCE era before the ISO epoch the year increases from 1 upwards as time goes backwards.
- proleptic-year - The proleptic year is the same as the year-of-era for the current era. For the previous era, years have zero, then negative values.
- month-of-year - There are 12 months in an ISO year, numbered from 1 to 12.
- day-of-month - There are between 28 and 31 days in each of the ISO month, numbered from 1 to 31. Months 4, 6, 9 and 11 have 30 days, Months 1, 3, 5, 7, 8, 10 and 12 have 31 days. Month 2 has 28 days, or 29 in a leap year.
- day-of-year - There are 365 days in a standard ISO year and 366 in a leap year. The days are numbered from 1 to 365 or 1 to 366.
- leap-year - Leap years occur every 4 years, except where the year is divisble by 100 and not divisble by 400.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final IsoChronology`	**`INSTANCE`**	Singleton instance of the ISO chronology.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`LocalDate`	**`date`**`(int prolepticYear, int month, int dayOfMonth)`	Obtains an ISO local date from the proleptic-year, month-of-year and day-of-month fields.
`LocalDate`	**`date`**`(Era era, int yearOfEra, int month, int dayOfMonth)`	Obtains an ISO local date from the era, year-of-era, month-of-year and day-of-month fields.
`LocalDate`	**`date`**`(TemporalAccessor tempor`	Obtains an ISO local date from another date-time object.
`LocalDate`	**`dateEpochDay`**`(long epochDay)`	Obtains an ISO local date from the epoch-day.
`LocalDate`	**`dateNow`**`()`	Obtains the current ISO local date from the system clock in the default time-zone.
`LocalDate`	**`dateNow`**`(Clock clock)`	Obtains the current ISO local date from the specified clock.
`LocalDate`	**`dateNow`**`(ZoneId zone)`	Obtains the current ISO local date from the

		system clock in the specified time-zone.
LocalDate	**dateYearDay**(int prolepticYear, int dayOfYear)	Obtains an ISO local date from the proleptic-year and day-of-year fields.
LocalDate	**dateYearDay**(Era era, int yearOfEra, int dayOfYear)	Obtains an ISO local date from the era, year-of-era and day-of-year fields.
long	**epochSecond**(int prolepticYear, int month, int dayOfMonth, int hour, int minute, int second, ZoneOffset zoneOffset)	Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.
IsoEra	**eraOf**(int eraValue)	Creates the chronology era object from the numeric value.
List<Era>	**eras**()	Gets the list of eras for the chronology.
String	**getCalendarType**()	Gets the calendar type of the underlying calendar system - 'iso8601'.
String	**getId**()	Gets the ID of the chronology - 'ISO'.
boolean	**isIsoBased**()	IsoChronology is an ISO based chronology, which supports fields in IsoFields, such as DAY_OF_QUARTER and QUARTER_OF_YEAR.
boolean	**isLeapYear**(long prolepticYear)	Checks if the year is a leap year, according to

		the ISO proleptic calendar system rules.
LocalDateTime	**localDateTime**(TemporalAccessor tempor	Obtains an ISO local date-time from another date-time object.
Period	**period**(int years, int months, int days)	Obtains a period for this chronology based on years, months and days.
int	**prolepticYear**(Era era, int yearOfEra)	Calculates the proleptic-year given the era and year-of-era.
ValueRange	**range**(ChronoField field)	Gets the range of valid values for the specified field.
LocalDate	**resolveDate**(Map<TemporalField, Long> fieldValues, ResolverStyle resolverSt	Resolves parsed ChronoField values into a date during parsing.
ZonedDateTime	**zonedDateTime**(Instant instant, ZoneId zone)	Obtains an ISO zoned date-time in this chronology from an Instant.
ZonedDateTime	**zonedDateTime**(TemporalAccessor tempor	Obtains an ISO zoned date-time from another date-time object.

Methods declared in class AbstractChronology

compareTo, equals, hashCode, toString

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Chronology

`epochSecond`, `getDisplayName`

Field Details

INSTANCE

`public static final IsoChronology INSTANCE`

Singleton instance of the ISO chronology.

Method Details

getId

`public String getId()`

Gets the ID of the chronology - 'ISO'.

The ID uniquely identifies the `Chronology`. It can be used to lookup the `Chronology` using `Chronology.of(String)`.

Specified by:

`getId` in interface `Chronology`

Returns:

the chronology ID - 'ISO'

See Also:

`getCalendarType()`

getCalendarType

`public String getCalendarType()`

Gets the calendar type of the underlying calendar system - 'iso8601'.

The calendar type is an identifier defined by the *Unicode Locale Data Markup Language (LDML)* specification. It can be used to lookup the `Chronology` using `Chronology.of(String)`. It can also be used as part of a locale, accessible via `Locale.getUnicodeLocaleType(String)` with the key 'ca'.

Specified by:

`getCalendarType` in interface `Chronology`

Returns:

the calendar system type - 'iso8601'

See Also:

`getId()`

date

```
public LocalDate date(Era era,
                      int yearOfEra,
                      int month,
                      int dayOfMonth)
```

Obtains an ISO local date from the era, year-of-era, month-of-year and day-of-month fields.

Specified by:

`date` in interface `Chronology`

Parameters:

`era` - the ISO era, not null

`yearOfEra` - the ISO year-of-era

`month` - the ISO month-of-year

`dayOfMonth` - the ISO day-of-month

Returns:

the ISO local date, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the type of `era` is not `IsoEra`

date

```
public LocalDate date(int prolepticYear,
                      int month,
                      int dayOfMonth)
```

Obtains an ISO local date from the proleptic-year, month-of-year and day-of-month fields.

This is equivalent to `LocalDate.of(int, int, int)`.

Specified by:

`date` in interface `Chronology`

Parameters:

`prolepticYear` - the ISO proleptic-year

`month` - the ISO month-of-year

dayOfMonth - the ISO day-of-month

Returns:

the ISO local date, not null

Throws:

DateTimeException - if unable to create the date

dateYearDay

```
public LocalDate dateYearDay(Era era,
                             int yearOfEra,
                             int dayOfYear)
```

Obtains an ISO local date from the era, year-of-era and day-of-year fields.

Specified by:

dateYearDay in interface Chronology

Parameters:

era - the ISO era, not null

yearOfEra - the ISO year-of-era

dayOfYear - the ISO day-of-year

Returns:

the ISO local date, not null

Throws:

DateTimeException - if unable to create the date

dateYearDay

```
public LocalDate dateYearDay(int prolepticYear,
                             int dayOfYear)
```

Obtains an ISO local date from the proleptic-year and day-of-year fields.

This is equivalent to LocalDate.ofYearDay(int, int).

Specified by:

dateYearDay in interface Chronology

Parameters:

prolepticYear - the ISO proleptic-year

dayOfYear - the ISO day-of-year

Returns:

the ISO local date, not null

Throws:

DateTimeException - if unable to create the date

dateEpochDay

`public LocalDate dateEpochDay(long epochDay)`

Obtains an ISO local date from the epoch-day.

This is equivalent to `LocalDate.ofEpochDay(long)`.

Specified by:

dateEpochDay in interface Chronology

Parameters:

epochDay - the epoch day

Returns:

the ISO local date, not null

Throws:

DateTimeException - if unable to create the date

date

`public LocalDate date(TemporalAccessor temporal)`

Obtains an ISO local date from another date-time object.

This is equivalent to `LocalDate.from(TemporalAccessor)`.

Specified by:

date in interface Chronology

Parameters:

temporal - the date-time object to convert, not null

Returns:

the ISO local date, not null

Throws:

DateTimeException - if unable to create the date

See Also:

ChronoLocalDate.from(TemporalAccessor)

epochSecond

```
public long epochSecond(int prolepticYear,
                        int month,
                        int dayOfMonth,
                        int hour,
                        int minute,
                        int second,
                        ZoneOffset zoneOffset)
```

Gets the number of seconds from the epoch of 1970-01-01T00:00:00Z.

The number of seconds is calculated using the year, month, day-of-month, hour, minute, second, and zoneOffset.

Specified by:

`epochSecond` in interface `Chronology`

Parameters:

`prolepticYear` - the year, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year, from 1 to 12

`dayOfMonth` - the day-of-month, from 1 to 31

`hour` - the hour-of-day, from 0 to 23

`minute` - the minute-of-hour, from 0 to 59

`second` - the second-of-minute, from 0 to 59

`zoneOffset` - the zone offset, not null

Returns:

the number of seconds relative to 1970-01-01T00:00:00Z, may be negative

Throws:

`DateTimeException` - if the value of any argument is out of range, or if the day-of-month is invalid for the month-of-year

Since:

9

localDateTime

```
public LocalDateTime localDateTime(TemporalAccessor temporal)
```

Obtains an ISO local date-time from another date-time object.

This is equivalent to `LocalDateTime.from(TemporalAccessor)`.

Specified by:

localDateTime in interface Chronology

Parameters:

temporal - the date-time object to convert, not null

Returns:

the ISO local date-time, not null

Throws:

DateTimeException - if unable to create the date-time

See Also:

ChronoLocalDateTime.from(TemporalAccessor)

zonedDateTime

```
public ZonedDateTime zonedDateTime(TemporalAccessor temporal)
```

Obtains an ISO zoned date-time from another date-time object.

This is equivalent to ZonedDateTime.from(TemporalAccessor).

Specified by:

zonedDateTime in interface Chronology

Parameters:

temporal - the date-time object to convert, not null

Returns:

the ISO zoned date-time, not null

Throws:

DateTimeException - if unable to create the date-time

See Also:

ChronoZonedDateTime.from(TemporalAccessor)

zonedDateTime

```
public ZonedDateTime zonedDateTime(Instant instant,
                                   ZoneId zone)
```

Obtains an ISO zoned date-time in this chronology from an Instant.

This is equivalent to ZonedDateTime.ofInstant(Instant, ZoneId).

Specified by:

zonedDateTime in interface Chronology

Parameters:

instant - the instant to create the date-time from, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

dateNow

```
public LocalDate dateNow()
```

Obtains the current ISO local date from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

`dateNow` in interface `Chronology`

Returns:

the current ISO local date using the system clock and default time-zone, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

```
public LocalDate dateNow(ZoneId zone)
```

Obtains the current ISO local date from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

`dateNow` in interface `Chronology`

Parameters:

zone - the zone ID to use, not null

Returns:

the current ISO local date using the system clock, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

```
public LocalDate dateNow(Clock clock)
```

Obtains the current ISO local date from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Specified by:

`dateNow` in interface `Chronology`

Parameters:

`clock` - the clock to use, not null

Returns:

the current ISO local date, not null

Throws:

`DateTimeException` - if unable to create the date

isLeapYear

```
public boolean isLeapYear(long prolepticYear)
```

Checks if the year is a leap year, according to the ISO proleptic calendar system rules.

This method applies the current rules for leap years across the whole time-line. In general, a year is a leap year if it is divisible by four without remainder. However, years divisible by 100, are not leap years, with the exception of years divisible by 400 which are.

For example, 1904 is a leap year it is divisible by 4. 1900 was not a leap year as
it is divisible by 100, however 2000 was a leap year as it is divisible by 400.

The calculation is proleptic - applying the same rules into the far future and far past. This is historically inaccurate, but is correct for the ISO-8601 standard.

Specified by:

`isLeapYear` in interface `Chronology`

Parameters:

`prolepticYear` - the ISO proleptic year to check

Returns:

true if the year is leap, false otherwise

prolepticYear

```
public int prolepticYear(Era era,
                         int yearOfEra)
```

Description copied from interface: `Chronology`

Calculates the proleptic-year given the era and year-of-era.

This combines the era and year-of-era into the single proleptic-year field.

If the chronology makes active use of eras, such as `JapaneseChronology` then the year-of-era will be validated against the era. For other chronologies, validation is optional.

Specified by:

`prolepticYear` in interface `Chronology`

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

Returns:

the proleptic-year

eraOf

```
public IsoEra eraOf(int eraValue)
```

Description copied from interface: `Chronology`

Creates the chronology era object from the numeric value.

The era is, conceptually, the largest division of the time-line. Most calendar systems have a single epoch dividing the time-line into two eras. However, some have multiple eras, such as one for the reign of each leader. The exact meaning is determined by the chronology according to the following constraints.

The era in use at 1970-01-01 must have the value 1. Later eras must have sequentially higher values. Earlier eras must have sequentially lower values. Each chronology must refer to an enum or similar singleton to provide the era values.

This method returns the singleton era of the correct type for the specified era value.

Specified by:

`eraOf` in interface `Chronology`

Parameters:

`eraValue` - the era value

Returns:

the calendar system era, not null

eras

```
public List<Era> eras()
```

Description copied from interface: `Chronology`

Gets the list of eras for the chronology.

Most calendar systems have an era, within which the year has meaning. If the calendar system does not support the concept of eras, an empty list must be returned.

Specified by:

`eras` in interface `Chronology`

Returns:

the list of eras for the chronology, may be immutable, not null

resolveDate

```
public LocalDate resolveDate(Map<TemporalField, Long> fieldValues,
                             ResolverStyle resolverStyle)
```

Resolves parsed `ChronoField` values into a date during parsing.

Most `TemporalField` implementations are resolved using the resolve method on the field. By contrast, the `ChronoField` class defines fields that only have meaning relative to the chronology. As such, `ChronoField` date fields are resolved here in the context of a specific chronology.

`ChronoField` instances on the ISO calendar system are resolved as follows.

- `EPOCH_DAY` - If present, this is converted to a `LocalDate` and all other date fields are then cross-checked against the date.
- `PROLEPTIC_MONTH` - If present, then it is split into the `YEAR` and `MONTH_OF_YEAR`. If the mode is strict or smart then the field is validated.
- `YEAR_OF_ERA` and `ERA` - If both are present, then they are combined to form a `YEAR`. In lenient mode, the `YEAR_OF_ERA` range is not validated, in

smart and strict mode it is. The ERA is validated for range in all three modes. If only the YEAR_OF_ERA is present, and the mode is smart or lenient, then the current era (CE/AD) is assumed. In strict mode, no era is assumed and the YEAR_OF_ERA is left untouched. If only the ERA is present, then it is left untouched.

- YEAR, MONTH_OF_YEAR and DAY_OF_MONTH - If all three are present, then they are combined to form a LocalDate. In all three modes, the YEAR is validated. If the mode is smart or strict, then the month and day are validated, with the day validated from 1 to 31. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first of January in the requested year, then adding the difference in months, then the difference in days. If the mode is smart, and the day-of-month is greater than the maximum for the year-month, then the day-of-month is adjusted to the last day-of-month. If the mode is strict, then the three fields must form a valid date.
- YEAR and DAY_OF_YEAR - If both are present, then they are combined to form a LocalDate. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first of January in the requested year, then adding the difference in days. If the mode is smart or strict, then the two fields must form a valid date.
- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and ALIGNED_DAY_OF_WEEK_IN_MONTH - If all four are present, then they are combined to form a LocalDate. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first of January in the requested year, then adding the difference in months, then the difference in weeks, then in days. If the mode is smart or strict, then the all four fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year and month, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the month.
- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and DAY_OF_WEEK - If all four are present, then they are combined to form a LocalDate. The approach
  is the same as described above for years, months and weeks in ALIGNED_DAY_OF_WEEK_IN_MONTH. The day-of-week is adjusted as the next or same matching day-of-week once the years, months and weeks have been handled.
- YEAR, ALIGNED_WEEK_OF_YEAR and ALIGNED_DAY_OF_WEEK_IN_YEAR - If all three are present, then they are combined to form a LocalDate. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first of January in the requested year, then adding the difference in weeks, then

in days. If the mode is smart or strict, then the all three fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the year.

- `YEAR`, `ALIGNED_WEEK_OF_YEAR` and `DAY_OF_WEEK` - If all three are present, then they are combined to form a `LocalDate`. The approach is the same as described above for years and weeks in `ALIGNED_DAY_OF_WEEK_IN_YEAR`. The day-of-week is adjusted as the next or same matching day-of-week once the years and weeks have been handled.

Specified by:

`resolveDate` in interface `Chronology`

Overrides:

`resolveDate` in class `AbstractChronology`

Parameters:

`fieldValues` - the map of fields to values, which can be updated, not null

`resolverStyle` - the requested type of resolve, not null

Returns:

the resolved date, null if insufficient information to create a date

Throws:

`DateTimeException` - if the date cannot be resolved, typically because of a conflict in the input data

range

```
public ValueRange range(ChronoField field)
```

Description copied from interface: `Chronology`

Gets the range of valid values for the specified field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

This method will return a result whether or not the chronology supports the field.

Specified by:

range in interface Chronology

Parameters:

field - the field to get the range for, not null

Returns:

the range of valid values for the field, not null

period

```
public Period period(int years,
                     int months,
                     int days)
```

Obtains a period for this chronology based on years, months and days.

This returns a period tied to the ISO chronology using the specified years, months and days. See Period for further details.

Specified by:

period in interface Chronology

Parameters:

years - the number of years, may be negative

months - the number of years, may be negative

days - the number of years, may be negative

Returns:

the ISO period, not null

isIsoBased

```
public boolean isIsoBased()
```

IsoChronology is an ISO based chronology, which supports fields in IsoFields, such as DAY_OF_QUARTER and QUARTER_OF_YEAR.

Specified by:

isIsoBased in interface Chronology

Returns:

true

Since:

19

See Also:

IsoFields

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.